Go2 SDK Development Guide

Search article titles

About Go2
Remote Controller
SDK Concepts 3
Application Development 7
Software Interface Services 19
Motion Services Interface V2.0
DDS Services Interface
Avoidance Services Interface
VuiClient Interface
RobotStateClient Interface
Motion Switcher Service Interface
Sports Services Interface
AI motion Services interface
Basic Services Interface
LiDAR Services Interface
Multimedia Services Interface
UWB Services Interface
Fault Services Interface
SLAM and Navigation Services Interfa
Aruco Recharge Service Interface
Odometer Services Interface
D1 Mechanical Arm Services Interface
ROS2 Services Interface
Python Services Interface
Case reference 8
DDS Communication
High Motion Control
Basic Motion Control
Deployment Framework
Light Control
Get Remote Control Status
RL Control Routine
Exapmle of motor errors
Debugging Specification
FAQ

Exapmle of motor errors

Last Updated On：2025-11-21 11:20:13

This article introduces how to use unitree_sdk2 to read the motor error state.

Motor errors

Motor errors refer to the motor entering a protection state or experiencing a malfunction. Potential motor errors are as follows:

Error type	Causes
Over current	The motor current exceeds the working range
Over voltage	Bus voltage exceeds 45V
Driver overheat	Temperature of the motor case or driver exceeds 90 ℃
Motor bus under voltage	Bus voltage below 11V
Winding overheat	Joint jamming or motor exceeding rated power for a long time resulting in winding temperature exceeding 140 ℃
Encoder abnormal	Motor encoder not available

note
When the motor enters the protection state (overcurrent, overvoltage, etc.), please turn off the robot first, rest for a period of time, and then turn it back on to restore normal operation. If the problem persists after multiple restarts, please contact technical support.

Users can get detial error information by analyzing the `lost` and `reserve` of **MotorState_.idl**. The explanation of the variables is as follows:

Variable	Describe
`lost`	Total lost packets in communication (Range：0-9999999999)
`reserve[0]`	Motor error code
`reserve[1]`	Current communication frequency (Range：0-800)

where motor error code `reserve[0]` is an integer variable. Each bit of `reserve[0]` binary corresponds to an error state. The relationship between each bit and the corresponding error state is as follows (1 indicates error):

bit	8	7	6	5	4	3	2	1	0
error_type	Communication interrupted	Reserved	Reserved	Encoder abnormal	Winding overheat	Motor bus under voltage	Driver overheat	Over voltage	Over current

For example：For example, `error_code = 6`, i. e. `error_code = 0b000000110`, means Over voltage and Driver overheat.

Example

In the example, `GetDetailMotorErrors` class is defined to parse the error code and report the corresponding error types.

```cpp
class GetDetailMotorErrors
{
private:
    struct MotorErrorState
    {
        std::string error_type;
        short int state;
    };

    std::vector<MotorErrorState> motor_errors = {
        {error_type : "Over current", state : 0},
        {error_type : "Over voltage", state : 0},
        {error_type : "Driver overheat", state : 0},
        {error_type : "Motor bus under voltage", state : 0},
        {error_type : "Winding overheat", state : 0},
        {error_type : "Encoder abnormal", state : 0},
        {error_type : "Reserved, Not used", state : 0},
        {error_type : "Reserved, Not used", state : 0},
        {error_type : "Communication interrupted", state : 0},
    };

public:
    GetDetailMotorErrors(){};
    ~GetDetailMotorErrors(){};
    void ReportErrors(int error_code)
    {
        for (int i = 0; i < motor_errors.size(); i++)
        {
            // Each bit of the error_code corresponds to a motor error type
            // For example, error_code = 6, i. e. error_code = 0b000000110, means Over voltage and Driver overhe
            motor_errors[i].state = (error_code & (1 << i)) >> i; // Get every bit of error_code.
            if (motor_errors[i].state > 0)
            {
                std::cout << motor_errors[i].error_type << ", ";
            }
        }
        std::cout << std::endl;
    };
};
```

This page contains content

- Motor errors
- Example

Helpful
Feedback

Then, use **low_level interface** to get the motor states, and use `GetDetailMotorErrors:: ReportErrors()` function to check motor errors.

```cpp
std::array<::unitree_go::msg::dds_::MotorState_, 20> motor_state_{};
GetDetailMotorErrors motor_errors_;

for (int i = 0; i < 20; i++)
{
    // Get motor state form low_state
    motor_state_[i] = low_state_.motor_state()[i];
    // Report motor errors
    motor_errors_.ReportErrors(motor_state_[i].reserve()[0]);
}
```

Full example is as follows:

```cpp
#include <iostream>
#include <unitree/robot/channel/channel_subscriber.hpp>
#include <unitree/idl/go2/LowState_.hpp>
#include <unitree/idl/go2/MotorState_.hpp>
#include <unitree/common/time/time_tool.hpp>
#include <unitree/common/thread/thread.hpp>

using namespace unitree::common;
using namespace unitree::robot;

#define TOPIC_LOWSTATE "rt/lowstate"

class GetDetailMotorErrors
{
private:
    struct MotorErrorState
    {
        std::string error_type;
        short int state;
    };

    std::vector<MotorErrorState> motor_errors = {
        {error_type : "Over current", state : 0},
        {error_type : "Over voltage", state : 0},
        {error_type : "Driver overheat", state : 0},
        {error_type : "Motor bus under voltage", state : 0},
        {error_type : "Winding overheat", state : 0},
        {error_type : "Encoder abnormal", state : 0},
        {error_type : "Reserved, Not used", state : 0},
        {error_type : "Reserved, Not used", state : 0},
        {error_type : "Communication interrupted", state : 0},
    };

public:
    GetDetailMotorErrors(){};
    ~GetDetailMotorErrors(){};
    void ReportErrors(int error_code)
    {
        for (int i = 0; i < motor_errors.size(); i++)
        {
            // Each bit of the error_code corresponds to a motor error type
            // For example, error_code = 6, i. e. error_code = 0b000000110, means Over voltage and Driver overhe
            motor_errors[i].state = (error_code & (1 << i)) >> i;
            if (motor_errors[i].state > 0)
            {
                std::cout << motor_errors[i].error_type << ", ";
            }
        }
        std::cout << std::endl;
    };
};

class Custom
{
public:
    explicit Custom() {}

    ~Custom() {}
    void Init();

private:
    void LowStateMessageHandler(const void *messages);
    void ReportMotorState();

private:
    unitree_go::msg::dds_::LowState_ low_state_{};
    std::array<::unitree_go::msg::dds_::MotorState_, 20> motor_state_{};

    ChannelSubscriberPtr<unitree_go::msg::dds_::LowState_> lowstate_subscriber_;

    /*MotorState logging thread*/
    ThreadPtr MotorStateReportThreadPtr;
    GetDetailMotorErrors motor_errors_;
};

void Custom::Init()
{
    /*create subscriber*/
    lowstate_subscriber_.reset(new ChannelSubscriber<unitree_go::msg::dds_::LowState_>(TOPIC_LOWSTATE));
    lowstate_subscriber_->InitChannel(std::bind(&Custom::LowStateMessageHandler, this, std::placeholders::_1), 1

    /*motor state logging thread*/
    MotorStateReportThreadPtr = CreateRecurrentThreadEx("log_motor_state", UT_CPU_ID_NONE, 500000, &Custom::Repo
}

void Custom::LowStateMessageHandler(const void *message)
{
    low_state_ = *(unitree_go::msg::dds_::LowState_ *)message;
}

void Custom::ReportMotorState()
{
    for (int i = 0; i < 20; i++)
    {
```

```
        motor_state_[i] = low_state_.motor_state()[i]; // Get MotorState from LowState
    }

    // Report states of the first motor (motor[0])
    std::cout << "Mode: " << motor_state_[0].mode() << ", "
              << "Position: " << motor_state_[0].q() << ", "
              << "Velocity: " << motor_state_[0].dq() << ", "
              << "Estimated torque: " << motor_state_[0].tau_est() << ", "

              // Motor packet loss information (range: 0-9999999999).
              << "Motor packet loss: " << motor_state_[0].lost() << ", "

              // Current motor error flag bits and communication frequency:
              // Array: 0- Motor error flag bits (Range: 0-255)
              // Array: 1- Current motor communication frequency (Range: 0-800)
              << "Error code: " << motor_state_[0].reserve()[0] << std::endl;

    motor_errors_.ReportErrors(motor_state_[0].reserve()[0]);
}

int main(int argc, const char **argv)
{
    if (argc < 2)
    {
        std::cout << "Usage: " << argv[0] << " networkInterface" << std::endl;
        exit(-1);
    }

    ChannelFactory::Instance()->Init(0, argv[1]);

    Custom custom;
    custom.Init();

    while (1)
    {
        sleep(10);
    }

    return 0;
}
```